Project Name	AI-Assisted Legacy Migration Knowledge Base
Online team meeting	https://fau.zoom-x.de/j/68356420940?pwd=yD6DYSba2jI1DTVMOvI3JmRchu9TuY.1
Production system (if any)	...
Test system (if any)	...
GitHub repository	https://github.com/amosproj/amos2026ws01-legacy-migrator
GitHub feature board	https://github.com/orgs/amosproj/projects/109
GitHub imp-squared backlog	https://github.com/orgs/amosproj/projects/113/views/1
Position of T-shirt logo	Upper Torso Right Small
Link to logo for black T-shirt	https://github.com/amosproj/amos2026ws01-legacy-migrator/blob/main/Deliverables/sprint-01/team-logo-blackTshirt.png
Link to logo for white T-shirt	https://github.com/amosproj/amos2026ws01-legacy-migrator/blob/main/Deliverables/sprint-01/team-logo-whiteTshirt.png
Additional materials	...
Team maling list	oss-amos-proj1@lists.fau.de

Last Name	First Name	GitHub User Name	Email Address
Fritz	**Josefina**	https://github.com/Josefinafr	josefina.fritz@fau.de
Barriga Vasquez	**Fernando**	https://github.com/fbarquez	f.barriga.vasquez@campus.tu-berlin.de
Sternberg	**Fiona**	https://github.com/Faoilthiama	fiona.sternberg@fau.de
Afzal Yousuf	**Ali**	https://github.com/3liAf	ali.yousuf@fau.de
Pohlitz	**Paul**	https://github.com/ppohlitze	p.pohlitz@campus.tu-berlin.de
Rubins	**Adrians**	https://github.com/AdriansRubins	il4707fu@zedat.fu-berlin.de
Kiesel	**Paul**	https://github.com/nudelisidle	paul.kiesel@fau.de
Ahm	**Raihan**	https://github.com/RaihanFAU	ahm.raihan@fau.de
Marques	**Rafael**	https://github.com/RafaelMcMarques	rafael.mc.marques@fau.de

		Product Owner					
#	Meeting Day	Review	Planning	Software Developer	Release Manager	Scrum Master	Comment
1	2026-10-14	Fernando	Josefina	Everyone else	N/A	Fiona	
2	2026-10-21	Josefina	Fernando	Everyone else	Ali	Fiona	
3	2026-10-28	Fernando	Josefina	Everyone else	Paul K.	Fiona	
4	2026-11-04	Josefina	Fernando	Everyone else	Adrians	Fiona	
5	2026-11-11	Fernando	Josefina	Everyone else	Paul P.	Fiona	
6	2026-11-18	Josefina	Fernando	Everyone else	Raihan	Fiona	
7	2026-11-25	Fernando	Josefina	Everyone else	Rafael	Fiona	Mid-term due
8	2026-12-02	Josefina	Fernando	Everyone else	Ali	Fiona	
9	2026-12-09	Fernando	Josefina	Everyone else	Paul K.	Fiona	
10	2027-01-11	Josefina	Fernando	Everyone else	Adrians	Fiona	
11	2027-01-18	Fernando	Josefina	Everyone else	Paul P.	Fiona	
12	2027-01-25	Josefina	Fernando	Everyone else	Raihan	Fiona	
13	2027-02-01	Fernando	Josefina	Everyone else	Rafael	Fiona	
14	2027-02-08	Josefina	Fernando	Everyone else	Ali	Fiona	Demo day!
15	2027-02-15	Fernando	Josefina	Everyone else	Paul K.	Fiona	Retrospective
Product owners, software developers, and Scurm Master are set and ideally don't change over time; the critical part is the Release Manager role you need to define here							

Goals	
	- Develop a useable and useful Prototype - Prototype should be compatible with MIT Software Licenses - Meet Requirements (that we don't know yet) from our industry partner - A good Teamwork - A Demo Version of the Prototype on Demo-day - Keep the Project realistic - Prototype should be easily demonstratable on Demo-day - Preferably locally testable (Ollama)
Meeting norms	
	-Meetings will be held in English -Be on time and come prepared > Punishments -Camera should be turned on (If it goes) -Let everyone speak and respect opinions and boundaries -If you are sick write a quick chat on WhatsApp and write the Professor
Working norms	
	- Ask for help if needed / Be helpful to others - Are you sick or unable to end your sprint work please let the POs know - AI Assistant Coding: Developer self should Review it first and the make a pull request - if AI used for git commit messages: PROPERLY Double check and remove "co-authored by Claude / etc." - Suggestion from Dirk Riehle: Required Review (If you create a merge request, you must review at least one another merge request) - When AI Agents are used: Create a system prompt file (for example .Vscode/copilot-instructions.md for GitHub-Copilot) and write down our coding norms there so that the AI adheres to them - Main Branch just touchable by the Release Manager that week – by Pull Request from Dev Branch - Dev branches with split branches (Title of backlog) – Pull request needs to be review - Breaking Change/Bugs handled by Release Manager and the one who caused it - Dirk Riehle said that collaborative coding must be annotated with a "Co-authored-by: Firstname Lastname <uni-email@uni.de>" at the end of a git commit message, one line per coworker. "Assigned to ..." flags on GitHub Projects work items will probably not be graded - Ensure local, easy reproducability and testability during development, right from the start. This is done by writing setup and start scripts, where it is necessary (instead of executing a module with long config files and 2329 env variables) - We all push into the develop branch then the release manager merges this then with the main branch
Coordination norms	
	- Planning Board Coordination - If you disagree about smth - discuss it with each other first via Discord/WA and maybe bring it with you to the team meeting session - Never move feature to "Feature Archive" (Finished) column without a review in "Awaiting Review" column first - When you assign someone else to a feature, notify them explicitly WhatsApp
Communication norms	
	- We set up a Discord Server - WhatsApp as main Communication Form - Happy Go Lucky: for twice a week Standup Email SDs (weekly for POs, SM) - Email with Wolfgang run by Fernando and Josefina - Feel free to text privately on WhatsApp if stuff comes up
Consideration norms	
	- Be patient – language differences, not same level of programming exp - respect each other's time - Be nice and kind to each other - Be open to constructive feedback and focus on solving problems rather than blaming each other - In planning poker always considere your peers thez might not have the same knowledge as you have
Cont. improvement norms	
	- No automating push into main and dev branch - Feature branch are named after tickets - After issue is resolved the feature branch for the ticket won't be deleted - Merge requests shall not be reviewed automatically - Everybody gets informed about resolved pull requests -Maybe leave a message on WA - If there is any issue that has to be resolved immediately find a time to talk together on Discord - Build process create a useful artefact - Artefacts are stored on Github - Vertical slices workstream
Rewards	
	-Get a T-shirt -Get a good grade -Online Celebration -Small Celebration for every release -A good Linkdin Profil : "What AMOS project taught me about B2B SAAS this week"
Sanctions	
	-If you're late to a team meeting, you have to dance when you join. -If you have to complete a penalty, you can choose to dance or sing. -https://allkaraoke.party/game/?playlist=English&song=radiohead-creep
Signatures	
Scrum Master	Fiona
Product owner	Fernando
Product owner	Josefina
Software developer	Paul
Software developer	Ali
Software developer	Adrian
Software developer	Rafael
Software developer	Raihan
Software developer	Paul

Product Vision	Project Mission
Enable software teams to migrate legacy systems with confidence by providing a persistent, connected knowledge base of legacy code, target code, domain knowledge and system dependencies. Through natural-language queries and up-to-date architecture views, teams can understand migration progress, preserve essential knowledge and make informed decisions about what to migrate or decommission next.	Develop and validate a prototype during the AMOS semester using a shareable reference project with a realistic migration history. Evaluate existing analysis tools, connect relevant information in a knowledge graph and demonstrate reliable natural-language answers about at least one migration step. Agree on measurable success criteria with the industry partner and define the scope of generated architecture views.

Term	Definition
Legacy System	The existing software system whose functionality is gradually migrated to a new system. Some components remain active during the transition.
Target System	The new system that gradually replaces legacy functionality. Its actual code is analysed to distinguish implemented functionality from planned work.
Knowledge Base	A persistent, versioned collection of connected information about legacy code, target code, domain knowledge and migration dependencies.
Knowledge Graph	A representation of information as nodes and relationships, connecting elements such as code components, bounded contexts and dependencies.
Abstract Syntax Tree (AST)	A structured representation of source code used to analyse elements such as classes, methods and fields.
Domain Model	A structured description of the business domain, including bounded contexts, business terminology, rules and events.
Bounded Context	A defined business boundary within which a particular domain model and its terminology apply. It can be linked to its implementations in both systems.
Ubiquitous Language	The shared business vocabulary used by domain experts and developers within a bounded context.
Coupling	A dependency between components or systems, such as direct calls, shared database tables or messaging routes. Remaining coupling can affect decommissioning decisions.
Decommissioning	Taking a legacy component out of operation after its functionality and remaining dependencies have been addressed.
Strangler Fig Migration	An incremental migration approach in which a new system gradually replaces the legacy system through parallel operation, completion and decommissioning.
Deterministic Ingestion	The stage that extracts and stores structural facts, such as code relationships and deployment information, without using an LLM.
LLM-Based Comprehension	The stage in which a large language model generates summaries and explanations from ingested information and stores them in the knowledge base.
Retrieval-Augmented Generation (RAG)	An approach that retrieves relevant knowledge-base information to support an LLM’s answer to a user’s question.
arc42	An architecture documentation framework. The project specifically targets its context, building block and deployment views.
Context View	An architecture view showing the system boundary, external actors, neighbouring systems and their interfaces.
Building Block View	An architecture view showing the static structure of components, services or modules. In this project, it shows legacy and target elements together with their status.
Deployment View	An architecture view showing where components run and how they connect through infrastructure such as databases and messaging systems.
Synthetic Reference System	A shareable example system with realistic migration scenarios, used to develop and validate the prototype without confidential client code or data.
Migration Step	A defined stage in the reference system’s migration history. Comparing consecutive steps reveals changes to code, deployments and dependencies.
Minimum Viable Product (MVP)	The initial prototype scope agreed with the industry partner. The kickoff discussion identifies answering natural-language questions about at least one migration step as the minimum goal.

Sprint #	Sprint goal
1	United Understanding
2	Research
3	Demonstrate a first end-to-end vertical slice that analyzes a reference repository and delivers a traceable result through the user interface
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

Sprint #	Story Points Realized
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
	PLEASE CREATE THE VELOCITY CHART ON A NEW TAB USING THE DATA FROM THIS TAB

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			0	0		
Sprints						
1			0	0	0	0
2			0	0	0	0
3			0	0	0	0
...				0		0
Features						
1						
2						
3						
		PLEASE CREATE THE BURNDOWN CHART ON A NEW TAB USING THE DATA FROM THIS TAB				

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			0	0		
Sprints						
1			0	0	0	0
2			0	0	0	0
3			0	0	0	0
...				0		0
Features						
1						
2						
3						
		PLEASE CREATE THE BURNDOWN CHART ON A NEW TAB USING THE DATA FROM THIS TAB				

#	Feature Definition of Done	Sprint Release Definition of Done	Project Release Definition of Done

Type	Link / reference

#	Context	Name	Version	Licence (SPDX)	Comment
1	container	python	3.12	PSF-2.0	Base image for pipeline and MCP server; code-review-graph needs Python >= 3.10
2	application	git	unpinned (resolve at first lockfile)	GPL-2.0-only	Host/base-image tool for git worktree checkouts; invoked as a separate process, not linked or modified
3	library	code-review-graph	2.3.9	MIT	Primary structural extractor (PyPI release 18 Sep 2026)
4	library	sqlglot	30.22.0	MIT	SQL read/write extraction (release 9 Oct 2026)
5	library	atlassian-python-api	5.0.5	Apache-2.0	Jira and Confluence connectors (Cloud and Server/DC); licence V, version from docs (S)
6	library	markdown-it-py	unpinned (resolve at first lockfile)	MIT	Markdown parsing (headings, links, code spans)
7	library	markdownify	unpinned (resolve at first lockfile)	MIT	Confluence storage format (XHTML) to markdown
8	library	mcp	unpinned (resolve at first lockfile)	MIT	MCP tool server SDK
9	library	pydantic	unpinned (resolve at first lockfile)	MIT	Canonical fact schema and validation
10	library	ollama (Python client)	unpinned (resolve at first lockfile)	MIT	Chat and embedding calls to the local model runtime
11	library	repowise	0.58.0	AGPL-3.0-or-later OR LicenseRef-repowise-commercial	Second extractor: workspaces and cross-repo contracts. Disable telemetry (opt-out). Version observed on PyPI earlier in the survey
12	library	neo4j (Python driver)	unpinned (resolve at first lockfile)	Apache-2.0	Loader and MCP queries
13	library	PyDriller	unpinned (resolve at first lockfile)	Apache-2.0	Git history mining (commits, changed files, issue keys)
14	container	ollama	0.40.9	MIT	Local LLM runtime. Sources disagree on latest (0.40.5 / 0.40.9 / 0.40.10); pinned 0.40.9. Cloud features must be disabled (offline configuration)
15	container	LibreChat	0.8.8	MIT	Chat UI (release 1 Oct 2026). Verify compose services at lock time
16	container	FerretDB	2.x (unpinned (resolve at first lockfile))	Apache-2.0	Replaces MongoDB (SSPL) as LibreChat's document store
17	library	DocumentDB PostgreSQL extension	unpinned (resolve at first lockfile)	MIT	Backend required by FerretDB 2.x; licence to verify
18	container	PostgreSQL	17 (minor unpinned (resolve at first lockfile))	PostgreSQL	Backend for FerretDB (both visions); in Vision A also graph + vector store
19	container	Neo4j Community Edition	calendar version unpinned (resolve at first lockfile)	GPL-3.0-only	Graph store and vector index; run unmodified as a separate server
20	application	code-maat	unpinned (resolve at first lockfile)	GPL-3.0-only	Co-change coupling, run as a separate JVM process; optional input
21	machine-learning-model	gpt-oss-20b	revision unpinned (resolve at first lockfile)	Apache-2.0	Chat model; Apache-2.0 plus the gpt-oss usage policy
22	machine-learning-model	Qwen3-Embedding-0.6B	revision unpinned (resolve at first lockfile)	Apache-2.0	Embedding model; check licence on the model card (Qwen licences differ per checkpoint)
23	library	pytest	unpinned (resolve at first lockfile)	MIT	Dev-only: tests (DoD)
24	application	cyclonedx-bom (cyclonedx-py)	unpinned (resolve at first lockfile)	Apache-2.0	Dev-only: generates the build SBOM from the lockfile
25	application	syft	unpinned (resolve at first lockfile)	Apache-2.0	Dev-only: SBOM for container images
26	application	pip-licenses	unpinned (resolve at first lockfile)	MIT	Dev-only: licence gate in CI
27	container	MongoDB Community	-	SSPL-1.0	Default LibreChat store; excluded (non-OSI), replaced by FerretDB
28	container	Meilisearch	-	MIT	Optional LibreChat search; disabled

#	Context	Name	Version	Licence (SPDX)	Comment					

#	Context	Name	Version	Licence (SPDX)	Comment					

#	Context	Name	Version	Licence (SPDX)	Comment					

#	Context	Name	Version	Licence (SPDX)	Comment					

Last Name	First Name	Value					
Fritz	**Josefina**	0		**3.00**	**OK**		
Barriga Vasquez	**Fernando**	0					
Sternberg	**Fiona**						
Afzal Yousuf	**Ali**						
Pohlitz	**Paul**	3		0	No size		
Rubins	**Adrians**	3		1	Trivial size		
Kiesel	**Paul**			2	Small size		
Ahm	**Raihan**	3		3	Medium size		
Marques	**Rafael**	3		5	Large size		
				8	Very large size		
				13	Too large (size)		
How to play planning poker							
1. Everyone type their number into their value field, don't hit return yet							
2. Someone, perhaps a product owner, count down 3.. 2.. 1..							
3. Then, everyone hit return to submit their value							